InSightX V3 — Claude Project Rules Summary

Consolidated view of `CLAUDE.md`, `.claude/rules/` and `.claude/skills/` · generated Oct 9, 2026 · updated Oct 15, 2026

> Part 1 is the always-loaded `CLAUDE.md`. Part 2 lists only what each rule file adds *beyond* `CLAUDE.md`, to avoid duplication. Part 3 is the skill table, and Part 4 says when to call each skill. Items marked **NEW** were added or changed after Oct 9 (decisions of Oct 13 and Oct 15, and the lightweight-dependency rule).

Part 1 — CLAUDE.md (always loaded, 133 lines)

Stack (target — not all installed yet)

- React 19 + React Compiler ON · TS strict · Vite · Tailwind v4 · Radix · Lucide · TanStack Query/Table/Virtual · react-hook-form + Zod + `@hookform/resolvers` · Zustand · cva/clsx/tailwind-merge · i18next · MapLibre + deck.gl · D3 · Vitest/Testing Library/Playwright
- No Redux. No CSS Modules, styled-components or Emotion.
- No new library without a note saying why.

NEW Keep it light

- The app carries heavy maps and graphs, so before adding any dependency check whether an existing library (e.g. Radix for toasts, dialogs, tooltips, dropdowns) or a native React/browser API already does the job
- Never add a library just for convenience
- If one is truly needed, pick the lightest and justify it briefly (see `performance.md`)

Import direction (CI-enforced)

- `shared/` never imports from `features/`
- Feature → feature only through `index.ts`, never a deep path
- Needed by two features → move to `shared/`, never import sideways
- `app/` imports anything; nothing imports `app/`

Naming

- Component — PascalCase, file matches component
- Hook — `useX`, camelCase file
- Other `.ts` — camelCase file
- Folder — kebab-case
- Callback prop — `onX`
- Handler in component — `handleX`, never also `onX`
- Boolean — `is` / `has` / `can` / `should`
- Type / interface — PascalCase, no `I` prefix
- Zod schema — `xSchema`
- Constants — SCREAMING_SNAKE

- Query key factory — `xKeys`

Components

- Keep logic out of JSX — calculations, data shaping, conditions longer than one ternary go above `return` or in a hook
- Pass handlers by reference, not called; arrow only to pass an argument, and not in virtualised rows
- Split container from presentation where a component has **both** a data source and real markup
- One exported component per file
- Props interface above the component, named `<Component>Props`
- No default exports except lazy route pages
- Under ~200 lines
- Never spread unknown props onto DOM outside `shared/ui`
- Early-return loading and error states
- Updater form when new state depends on old; never mutate, in state or in a store

State ladder — stop at the first row that fits

- Server data → TanStack Query
- Shareable / bookmarkable → the URL
- One component → `useState`
- Related values with clear actions → `useReducer`
- Rarely-changing, app-wide → Context
- Frequently-changing, cross-feature → Zustand (selectors, always)
- Form fields → react-hook-form
- **Above all:** server data has one owner, the query cache
- Context is not a state manager — every consumer re-renders

useEffect is a last resort

- Legitimate: graph/map instances · WebSocket/SSE · window/document listeners · driving a non-React library · observers
- Every setup returns a cleanup
- Instead: fetching → `useQuery` · deriving → compute in render · reset on prop change → `key` · syncing two states → delete one · on click → event handler · toast → `onSuccess`

Always

- No hardcoded user-facing strings — `t()`, structured keys, ICU plurals, one namespace per feature
- Logical CSS properties only (`ms-` / `me-` / `ps-` / `pe-` / `start-` / `end-`); canvas stays `dir="ltr"`
- No raw colours — tokens only; status/priority colours from a mapping function; `cva` + `cn()`; no arbitrary values
- Dates/numbers through `Intl`, centralised in `shared/lib/format.ts`
- Check `shared/hooks` before writing a utility hook
- Every route lazy; no heavy library in the entry chunk
- Compiler is on — no hand-written `useMemo` / `useCallback` / `React.memo` except the listed exceptions
- Errors in one place: shared error boundary + mutation toast; no `try/catch` in components

Definition of done

- 1. Types strict, no `any` · 2. Loading/empty/error states · 3. Keyboard accessible, focus visible · 4. Arabic/RTL + dark mode · 5. No new library without a note · 6. Lists over 100 rows virtualised · 7. Strings translated

Part 2 — .claude/rules/ (additions only)

`architecture.md`

- The `src/` tree: `app/` · `shared/{ui,lib,hooks,types}` · `features/<domain>/{api,components,hooks,model,pages}` + `index.ts`
- Enforce rules 1–2 with `eslint-plugin-boundaries` or import restricted zones — an unenforced boundary lasts four weeks
- NEW `@/*` → `src/*` alias is configured in `tsconfig.app.json` and `vite.config.ts`
- NEW Router: **React Router v7** (`react-router`, decided Oct 13). Routes in `app/router/router.tsx`; lazy pages and chunk loaders in `app/router/lazyPages.ts`, so the sidebar prefetches the same chunk on hover
- NEW Each route has its own `errorElement`; URL state goes through `shared/hooks/useUrlState`, built on `useSearchParams`
- NEW Sixteen sidebar modules → every feature registers exactly one lazy route
- Build order: `shared/ui` + API client → Case table → Create Case form + intake tabs

`state-management.md`

- Why no Redux: "global state" here is mostly server state; Redux = two caches for the same data
- One Zustand store per concern, in the feature's `model/`
- Always subscribe with a selector — never `useStore()` bare
- Never mutate — produce a new value, including in Immer-style updates
- NEW Persistence: Zustand `persist` with `createIndexedDbStorage()` from `shared/lib/indexedDbStorage.ts` (idb-keyval, decided Oct 15). Stores structured clones, so `File`s persist
- NEW Use `skipHydration` and hydrate on demand (see `ensureDraftsHydrated`)

`api-layer.md`

- Three layers: `apiClient.ts` → `*.api.ts` (no React) → `*.queries.ts`; components import layer 3 only
- NEW The client is **native** `fetch` (decided Oct 13): base URL, auth header, errors normalised into `ApiError`, `AbortSignal` support. Nothing else calls `fetch`
- NEW Transport swappable with `setApiTransport()` — the in-browser mock server in `src/mocks/` uses it while there is no backend (`VITE_USE_MOCKS=true`)
- Query keys from a factory, never inline
- Every query forwards `signal`
- Zod-validate responses at the boundary for anything complex

- `staleTime` set deliberately per data type (reference = minutes, lists = seconds)
- `placeholderData: keepPreviousData` on paginated tables
- Mutations invalidate; they don't rewrite the cache unless optimistic
- Overlay queries mount only when the overlay opens

`hooks.md`

- Extract when logic repeats, or >~30 lines above `return`
- Don't extract a stateless one-off — that's a function for `model/` or `lib/`
- One feature → feature `hooks/`; two+ → `shared/hooks/`; wraps a server call → `api/`
- Return an **object** past two values
- Build early: `useDebounce` · `useDebouncedValue` · `useUrlState` · `useDisclosure` · `useUnsavedChangesGuard` · `useMediaQuery` · `usePermission` · `useKeyboardShortcut` · `useConfirm`
- Standardise: `useId` · `useTransition` · `useDeferredValue` · `useSyncExternalStore` · `useRef` · `useImperativeHandle` · error boundaries · `<Suspense>`

`tables-filters-search.md`

- One `DataTable` in `shared/ui/data-table/` — never a second
- It owns: headers, sorting UI, selection, expansion, empty state, skeleton, pagination, column visibility, sticky header, virtualisation. The feature provides `columns.tsx` + a query hook
- Column defs in their own file — data, not JSX in a component
- Server-side sorting/filtering/paging by default; client-side only under ~500 rows
- Virtualise past 100 visible rows
- `getRowId` returns the real entity id, never the index
- Cell renderers are small named components
- Filter state lives in the URL, always
- One filter hook per table, and it's the single input to the query key
- Quick tabs are filter presets, not a separate feature
- Tab counts come from the server in **one** call
- Three searches that must not share code: global ⌘K · in-table (URL, 300 ms) · local `filter()`
- Debounce the value, not the request — never `setTimeout` in a `useEffect`

`forms.md`

- react-hook-form + Zod + `@hookform/resolvers`; never `useState` per field
- Schema is the contract — written once in `model/`, type via `z.infer`
- Field components in `shared/ui/form/`; features never call `register()` by hand
- Three save actions: Create (full validation → mutate → invalidate → navigate → toast) · Save as draft (`schema.partial()`, stay) · Discard (confirm if dirty)
- Disable submit while pending, pending state on the button
- Server field errors mapped back with `setError`
- Autosave drafts on a debounce; never autosave a final submit
- Block navigation when dirty via the shared guard

- `reset()` after create, don't remount
- Never unmount a collapsed section holding data — hide with CSS

`overlays.md`

- Radix wrapped once in `shared/ui/`; features never import `@radix-ui/*` or build their own modal
- Pick by job: Dialog · Sheet · Popover · DropdownMenu · Tooltip (labels only) · Toast
- Routed if linkable/refreshable/in history; local via `useDisclosure` if transient
- Never nest a dialog in a dialog — make it a step
- One shared promise-returning `useConfirm()`, backed by `shared/ui/ConfirmDialog`
- NEW Toasts: **Radix Toast** (decided Oct 15). Fire `toast.success(…)` from `shared/lib/toast.ts` in mutation callbacks, rendered by `shared/ui/Toaster`. Messages are translated in the mutation hook
- Portal root carries the correct `dir`

`styling.md`

- Tailwind v4 only
- Tokens as CSS variables in `@theme`; never a raw colour in a component
- `cva` for variants, `cn()` for conditional classes; never template-literal class strings
- No arbitrary values outside prototypes — needed twice means it's a token
- Status/priority/entity colours from a mapping function
- Logical properties only
- NEW Dark mode via a `.dark` **class on** `<html>` (decided Oct 13), driven by one `ThemeProvider`. `index.css` redefines the same tokens under `.dark`, so components never need `dark:` variants
- Features compose `shared/ui`, don't restyle it

`i18n-rtl.md`

- No hardcoded strings; ESLint rule for literal JSX text
- One namespace per feature, lazy-loaded with the route
- Structured keys, not sentences
- ICU plurals — Arabic has six forms
- `dir` set once on `<html>`; components never check language to pick a margin
- Canvas containers stay `dir="ltr"`
- Dates/numbers via `Intl`, centralised
- Self-host the Arabic font — on-prem can't call Google Fonts

`performance.md`

- React Compiler enabled via `babel-plugin-react-compiler` in `vite.config.ts`
- Keep manual memoisation only for: expensive computation · values passed to non-React libraries · costly effect deps · context values. `React.memo` for hundreds of rows
- Every route lazy; every heavy library behind a lazy route or dynamic import
- Prefetch on intent (sidebar hover)

- i18n namespaces per module
- CI budgets, build fails: 250 KB initial JS gzipped · 400 KB per lazy chunk · 100 unvirtualised DOM rows · 2,000 timeline markers before Canvas
- NEW **Lightweight dependencies**, checked in order: (1) can an existing library or native API do it? (Radix, TanStack, cva, Lucide, i18next, Zod; `Intl`, `fetch`, `AbortController`, observers, CSS) · (2) only for convenience? then don't add it · (3) only if the existing option genuinely falls short, recommend the lightest alternative
- NEW Evaluate candidates on: gzipped size · tree-shaking (ESM, `sideEffects`) · can it sit behind a lazy route · runtime cost · overlap with installed libraries · effect on map/graph chunks and budgets
- NEW A justified dependency gets a short note in the PR or reply: why existing options fall short, and why this one is the best trade-off
- NEW No speculative abstractions or duplicated utilities, but don't trade away readability to save lines. Check `shared/lib` and `shared/hooks` first
- Skeletons not spinners · rAF for animation · debounced inputs · Web Workers for graph layout and CSV/CDR parsing · IndexedDB for workspaces

`heavy-visual-modules.md`

- Wrapper pattern: empty `<div ref dir="ltr">` → `useEffect` with empty deps creates + destroys → instance in `useRef` → data pushed imperatively → library events become React callbacks
- One graph and one map instance alive at a time; unmount means destroy
- `dir="ltr"` on every canvas container, even in Arabic
- Pin deck.gl and MapLibre exactly; deck.gl in overlaid mode
- **Graphology results describe the loaded subgraph only** — any path/centrality/"no connection" UI must say so. The most important rule in the section
- Timeline switches SVG → Canvas above ~2,000 markers
- Own error boundary per visualisation

`testing.md`

- Vitest: pure logic, schemas, formatters, reducers — everything
- Vitest + TL: hooks with real logic
- Testing Library: shared `ui` + complex forms only
- Playwright: create case, filter/find, open, switch language
- Graph/map: smoke only — mounts, renders, destroys cleanly
- CI gates: TS strict no `any`, no bare `@ts-ignore` · ESLint (hooks, boundaries, literal strings) · bundle budgets

`open-questions.md`

- Not rules — the register of what's undecided, unspecified, conflicting, and missing from the repo
- NEW Resolved and moved into the rules: router (React Router v7), API client (native `fetch`), dark mode (`.dark` class), toasts (Radix Toast), IndexedDB storage (idb-keyval), `@/*` alias, TS `strict`
- Still open: graph library (Sigma.js *or* Cytoscape) · boundary enforcement (`eslint-plugin-boundaries` *or* `eslint-plugin-import` zones)

- Not specified yet: auth scheme · full token palette · backend API shape (the mock assumes a 422 `{ fieldErrors }` body) · real priority service levels (`PRIORITY_DUE_DAYS` is an assumption)
- Repo gaps: MapLibre, deck.gl, D3, Vitest, Testing Library, Playwright not installed · no boundary or literal-text ESLint rules · no `size-limit` budget check

Part 3 — .claude/skills/ (11 skills, recipe + hard constraints)

Skill	Triggers on	The constraints it enforces
`add-api-endpoint`	fetch, endpoint, query, mutation	Three layers · key factory, never inline · forward `signal` · no `fetch` outside `apiClient` (native fetch) · no `try/catch` in components
`build-form`	form, create/edit page, validation	Schema first · never `useState` per field · never `register()` by hand · don't unmount sections with data · `reset()` not remount · never autosave a submit
`add-data-table`	table, list page, grid	One `DataTable` · columns in their own file · named cell components · real `getRowId` · virtualise past 100 · headers are translation keys
`add-table-filters`	filter, in-table search, sort	URL only, via `useUrlState` (reference: `useCaseFilters`) · one hook per table · debounce the value · reset `page` on filter change · one call for tab counts
`new-feature-module`	new module, scaffold	Folder shape · deliberate `index.ts` · lazy route in `router.tsx` + `lazyPages.ts` (the one allowed default export) · own i18n namespace · prefetch on hover · boundary check
`add-dialog`	dialog, modal, panel, confirm	Pick the right type · routed vs local · no nested dialogs · no interactive tooltips · lazy overlay queries · toasts from mutations (Radix `Toaster`) · `useConfirm()`
`design-system-component`	badge, button, chip, shared component	Tokens first · `cva` + `cn()` · no arbitrary values · mapping function not JSX ternary · logical properties · props spread allowed only here
`multi-tab-workspace`	tabs, drafts, open multiple	Only the active tab mounted · store is truth, form is a copy (debounced 500 ms) · IndexedDB not localStorage · per-tab dirty state · selectors always
`canvas-library-wrapper`	map, graph, timeline, canvas	ref + empty deps + cleanup · instance in `useRef` · imperative updates · single instance · `dir="ltr"` · loaded-subgraph caveat · own error boundary
`write-custom-hook`	hook, extract logic	Is it even a hook? · check `shared/hooks` first · placement table · object return · no `useQuery` wrapper in `hooks/`
`review-project-instructions`	after a feature lands	Report, never edit · theory→real-code swap · move resolved open questions out · the "corrected twice" signal · a violated valid rule is a code finding

NEW Part 4 — When to call each skill

> Claude always sees each skill's name and description and loads a skill automatically when a prompt matches it. Typing `/skill-name` forces it, which is the reliable choice when you already know a skill applies. CLAUDE.md and every `.claude/rules/*.md` file load in full each session; a skill's full `SKILL.md` loads only when that skill runs.

Skill	Call it when…
`/new-feature-module`	Starting a new sidebar module or domain (entities, graph, map, ETL…)
`/add-api-endpoint`	Any new fetch, query, mutation or cache invalidation
`/add-data-table`	Any list, grid or results table, or new columns, row expansion or virtualisation
`/add-table-filters`	Filters, quick-tabs, sorting, in-table search, URL state
`/build-form`	Create or edit pages, validation, save-as-draft, submit or discard
`/multi-tab-workspace`	Several drafts or workspaces open at once that survive reload (Zustand + IndexedDB)
`/add-dialog`	Modal, sheet or side panel, popover, dropdown, tooltip, confirm
`/design-system-component`	A new or changed **shared** primitive in `shared/ui` (button, badge, chip, card…), variants or tokens. Not for styling a feature page
`/write-custom-hook`	Moving logic out of a component, or adding to `shared/hooks`
`/canvas-library-wrapper`	Map, graph, link analysis, timeline, or any WebGL/canvas library
`/review-project-instructions`	**Once, after a feature is finished**, before you commit or open a PR. Not on every prompt

Typical order for a full feature: `new-feature-module` → `add-api-endpoint` → `add-data-table` + `add-table-filters` → `build-form` / `add-dialog` as needed → `design-system-component` only if a new shared primitive is needed → `review-project-instructions`.

Two things worth noticing from seeing it all at once: the definition-of-done checklist is the only rule intentionally duplicated (`CLAUDE.md` + `testing.md`); everything else has exactly one home. Three rules appear as constraints in four or more skills (*URL-backed state*, *no raw colours / tokens only*, and *translate every string*), which is why they're the ones promoted into `CLAUDE.md`'s "Always" block.